STAT XXXX Generate AI

Chi-Kuang Yeh

Oct 21, 2026

Contents

Course outline **3**
Road map . . . 4
Part I · From statistical learning to attention . . . 4
Part II · Transformers and generative models . . . 6
Part III · Reliable systems and agents . . . 7
Optional directions after the core . . . 9

Learn, teach, and build with Python **10**
1 · Choose how to study . . . 10
2 · Create a Python environment . . . 10
3 · Open a notebook . . . 11
4 · Build the website and PDF . . . 11
5 · Understand the source files . . . 12
6 · Publish a reviewed build . . . 13

The statistical lens **14**
A translation guide . . . 15
Three equations to keep in view . . . 15
Questions for every paper or experiment . . . 16

Reading guide **17**
First-pass resources . . . 17
Foundational papers by module . . . 17
A statistician's reading note . . . 18

1 · From probability models to generative AI **19**
1.1 · What is a generative model? . . . 19
1.2 · The statistical workflow we will preserve . . . 20
1.3 · The Python you need to start . . . 21
1.4 · Fit a categorical model by maximum likelihood . . . 23
1.5 · Log loss, cross-entropy, and perplexity . . . 23
1.6 · Zero counts, smoothing, and a Bayesian connection . . . 24
1.7 · Generation is a sampling operation . . . 25
1.8 · Temperature changes the distribution used for sampling . . . 27
1.9 · From independent tokens to a language model . . . 29
1.10 · Generate a sequence one decision at a time . . . 32
1.11 · Evaluate prediction separately from generation . . . 33

1.12 · Common Python problems and how to diagnose them . . . 36
1.13 · Exercises . . . 36
1.14 · Worked solutions . . . 37
1.15 · What this chapter gives us . . . 39

2 · Python and neural network foundations **41**

3 · Tokens, embeddings, and representations **42**

4 · Attention from first principles **43**

5 · Assembling the transformer **44**

6 · Language model training, prompting, and decoding **45**

7 · Latent variables, VAEs, GANs, and flows **46**

8 · Diffusion and score-based generation **47**

9 · Fine-tuning, LoRA, and preference learning **48**

10 · Retrieval-augmented generation (RAG) **49**

11 · Evaluation, calibration, and uncertainty **50**

12 · Agents: tools, state, and decisions **51**

13 · Agent reliability and experimental design **52**

14 · Capstone: a GenAI system for a statistician **53**

First Python lab: probabilities to generation **54**
1 · Represent a probability distribution . . . 54
2 · Evaluate log loss . . . 54
3 · Change the sampling temperature . . . 55
4 · Generate and inspect a Monte Carlo sample . . . 56
Try it yourself . . . 57
Bridge to attention . . . 57

Course outline

STAT XXXX Generate AI

This sequence assumes a strong statistics background and begins at an introductory level in deep learning and GenAI. Modules are learning units, not calendar weeks. Work through them at a comfortable pace, returning to the mathematics when it clarifies an experiment. All proposed implementations use **Python**, progressing from NumPy to PyTorch and then selected pretrained-model tools.

Road map

Module	Topic	Statistical connection	Evidence of learning
1	*Generative AI as probabilistic modeling* (page 19)	Density estimation, likelihood, risk	Derive and sample a tiny language model
2	*Python and neural network foundations* (page 41)	GLMs, optimization, regularization	Fit a small neural predictor
3	*Tokens and embeddings* (page 42)	Feature maps, dimension reduction	Compare two text representations
4	*Attention from first principles* (page 43)	Adaptive weighted averages	Implement and inspect attention
5	*The transformer* (page 44)	Compositional representations	Trace a decoder block
6	*Training and using language models* (page 45)	Conditional likelihood, sampling	Compare decoding rules
7	*Latent-variable generative models* (page 46)	Variational inference, change of variables	Derive an ELBO and simulate
8	*Diffusion and score-based modeling* (page 47)	Denoising, stochastic processes	Run a toy noise/recovery experiment
9	*Adaptation and alignment* (page 48)	Low-rank constraints, paired comparisons	Compare adaptation strategies
10	*Retrieval-augmented generation* (page 49)	Conditional prediction, selection	Build and inspect a small RAG pipeline
11	*Evaluation and uncertainty* (page 50)	Experimental design, calibration	Produce an evaluation report
12	*Agents and tool use* (page 51)	Sequential decisions, stopping	Build a bounded single agent
13	*Reliable agent experiments* (page 52)	Dependence, ablations, reliability	Compare agent and workflow baselines
14	*A statistician's GenAI project* (page 53)	Reproducible empirical research	Deliver a small teaching or research system

Part I · From statistical learning to attention

1 · Generative AI as probabilistic modeling

Question. What does a generative model learn, and what does it mean to generate a new observation?

- **Learn:** distinguish joint, conditional, and discriminative models; use the chain rule to factor a text distribution; connect cross-entropy to negative log likelihood.
- **Statistical lens:** density estimation, empirical risk, smoothing, and held-out log loss.
- **Python practice:** estimate a tiny categorical or bigram model, sample sequences, and compare training and held-out loss. *Chapter 1* (page 19) develops the full example, including a bigram language model.

- **Self-check:** explain why fluent sampled text need not be factually correct, and why zero-probability events cause problems for log loss.

2 · Python and neural network foundations

Question. What changes when a regression model learns its own nonlinear features?

- **Learn:** arrays, shapes, indexing, broadcasting, functions, and reproducible random generators; then tensors, automatic differentiation, minibatches, and a PyTorch training loop.
- **Statistical lens:** linear/logistic regression, basis expansions, regularization, optimization, overfitting, and validation.
- **Python practice:** fit logistic regression and a small multilayer perceptron to the same simulated problem; record train/validation curves and compare against a simple baseline.
- **Self-check:** distinguish the model, loss, optimizer, and data split; check a derivative numerically before trusting autodiff.

3 · Tokens, embeddings, and representations

Question. How does text become something a numerical model can use?

- **Learn:** tokenization, vocabulary, one-hot representations, embedding lookup, positional information, cosine similarity, and contextual representations.
- **Statistical lens:** design matrices, learned feature maps, matrix factorization, and identifiability.
- **Python practice:** tokenize a small public text collection and compare bag-of-words with embedding-based similarity. Inspect examples where similarity is misleading.
- **Self-check:** explain why a token is not necessarily a word and why a two-dimensional embedding plot is not evidence of causal structure.

4 · Attention from first principles

Question. How can a representation select and combine information relevant to a query?

- **Learn:** queries, keys, values, scaled dot products, row-wise softmax, self-attention, cross-attention, causal masks, and multiple heads.
- **Statistical lens:** a query-dependent normalized weighted average; compare with kernel smoothing without claiming exact equivalence.
- **Python practice:** calculate a three-token attention example by hand, implement it in NumPy, verify dimensions and row sums, and plot how weights change when keys change.
- **Self-check:** explain the role of the mask and the $\sqrt{d_k}$ scaling; describe why attention weights alone are not a faithful explanation of a prediction.
- **Reading:** Attention Is All You Need[1], especially the architecture and attention equations.

[1] https://arxiv.org/abs/1706.03762

Part II · Transformers and generative models

5 · Assembling the transformer

Question. How does one attention calculation become a sequence model?

- **Learn:** residual connections, normalization, feed-forward layers, positional encodings, encoder/decoder distinctions, and a decoder-only forward pass.
- **Statistical lens:** a composition of fitted feature transformations with shared parameters and a conditional prediction head.
- **Python practice:** implement a tiny decoder block in PyTorch; track every tensor's dimensions and use a causal-mask check to show future tokens cannot influence earlier outputs.
- **Self-check:** draw the route from token IDs to next-token probabilities, and distinguish the original encoder-decoder transformer from a decoder-only language model.

6 · Language model training, prompting, and decoding

Question. What is learned in the weights, and what changes when we provide a prompt?

- **Learn:** next-token prediction, teacher forcing, context windows, pretraining versus inference, in-context examples, greedy decoding, temperature, top-k, and top-p.
- **Statistical lens:** conditional likelihood, distribution shift, and transformations of a sampling distribution. In-context behavior can be studied statistically without assuming the model performs exact Bayesian inference.
- **Python practice:** train a very small character model or use a small pretrained model; vary one decoding choice at a time on fixed prompts and repeat runs.
- **Self-check:** explain why perplexity is not a universal measure of usefulness and why prompting is not parameter estimation by gradient descent at inference time.

7 · Latent variables, VAEs, GANs, and flows

Question. What alternatives to sequential factorization can represent a complex distribution?

- **Learn:** the encoder/decoder view of a VAE, the evidence lower bound (ELBO), reparameterization, and a conceptual comparison with adversarial learning and invertible flows.
- **Statistical lens:** latent-variable models, variational inference, density-ratio ideas, and change of variables. Compare objectives, sampling, and access to likelihoods.
- **Python practice:** derive the ELBO and implement a small VAE on synthetic two-dimensional data. Treat GANs and flows as comparison topics on the first pass.
- **Self-check:** separate the latent prior, approximate posterior, and observation model; explain why matching a few summary statistics cannot establish distributional equivalence.
- **Reading:** Auto-Encoding Variational Bayes[2].

[2] https://arxiv.org/abs/1312.6114

8 · Diffusion and score-based generation

Question. How can learning to remove noise support generation from noise?

- **Learn:** a forward noising process, learned reverse transitions, denoising objectives, score functions, and conditional generation; preview image and multimodal applications.
- **Statistical lens:** stochastic processes, denoising estimation, and the score $\nabla_x \log p(x)$, which differs from a likelihood score with respect to model parameters.
- **Python practice:** simulate progressive Gaussian noising on a mixture distribution, then study a small learned denoiser and the effect of the noise schedule.
- **Self-check:** distinguish the known forward process from the fitted reverse process and separate training from sampling.
- **Reading:** Denoising Diffusion Probabilistic Models[3]. Continuous-time formulations are an optional extension.

9 · Fine-tuning, LoRA, and preference learning

Question. How do we adapt an existing model, and whose preferences are represented?

- **Learn:** supervised fine-tuning, parameter-efficient updates, LoRA, instruction data, preference data, the RLHF pipeline, and the basic DPO idea.
- **Statistical lens:** transfer learning, regularization, low-rank parameter updates, and Bradley–Terry-style paired comparisons. Keep model quality separate from preference-label quality.
- **Python practice:** fit a toy paired-comparison model and compare a full matrix update with a low-rank update. Small-model fine-tuning is optional after the conceptual exercise.
- **Self-check:** explain what changes under prompting, RAG, and fine-tuning; identify label noise and selection bias in preference data.
- **Readings:** LoRA[4] and DPO[5].

Part III · Reliable systems and agents

10 · Retrieval-augmented generation (RAG)

Question. How can a model answer using a specified collection of documents?

- **Learn:** document chunking, indexing, retrieval, reranking, context assembly, citation, and abstention when evidence is insufficient.
- **Statistical lens:** prediction conditional on selected evidence, retrieval error, measurement error, and the interaction between pipeline components.
- **Python practice:** build a small pipeline over public statistical notes. Inspect retrieved passages, compare with a keyword baseline, and test answerable and unanswerable questions separately.

[3] https://arxiv.org/abs/2006.11239
[4] https://arxiv.org/abs/2106.09685
[5] https://arxiv.org/abs/2305.18290

- **Self-check:** identify whether an error arose in retrieval, evidence selection, or generation. A citation does not by itself show that its source supports the claim.
- **Reading:** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[6].

11 · Evaluation, calibration, and uncertainty

Question. What evidence would justify saying one system is better than another?

- **Learn:** task-specific metrics, held-out evaluation, data contamination, human annotation, LLM-as-a-judge limitations, calibration, abstention, and an introduction to conformal methods.
- **Statistical lens:** estimands, sampling units, paired comparisons, bootstrap uncertainty, clustered dependence, and distribution shift. Conformal coverage needs an explicit target and assumptions such as exchangeability; it is not an automatic guarantee of free-text truthfulness.
- **Python practice:** compare two fixed systems on the same held-out questions; bootstrap at the question level, inspect judge disagreement, and draw a risk-versus-coverage curve for abstention.
- **Self-check:** distinguish token probabilities, answer confidence, and factual correctness; state the population to which a reported interval applies.
- **Reading:** A Gentle Introduction to Conformal Prediction[7].

12 · Agents: tools, state, and decisions

Question. What makes a system an agent, beyond producing a text response?

- **Learn:** a model–action–observation loop, tool schemas, state, memory, structured outputs, retries, and stopping rules; distinguish a fixed workflow from adaptive action selection.
- **Statistical lens:** sequential decision making, partial observation, expected utility, and the value of information. Learn the basic state/action/reward vocabulary without requiring a reinforcement learning course first.
- **Python practice:** build a single agent with two narrow tools, such as a calculator and document lookup. Validate arguments, log actions and observations, and enforce a small step budget.
- **Self-check:** explain when a fixed Python function or workflow is sufficient and why tool access does not imply reliable tool use.
- **Reading:** ReAct[8]; study observable actions and results without treating generated reasoning text as a faithful record of internal computation.

13 · Agent reliability and experimental design

Question. Does an agent improve outcomes enough to justify its additional complexity?

- **Learn:** trajectory-level evaluation, failure taxonomies, repeated trials, cost/latency tradeoffs, memory ablations, prompt injection through retrieved text, and human approval for consequential actions.

[6] https://arxiv.org/abs/2005.11401
[7] https://arxiv.org/abs/2107.07511
[8] https://arxiv.org/abs/2210.03629

- **Statistical lens:** dependence within a task trajectory, factorial comparisons, uncertainty in success rates, and adaptive selection bias.
- **Python practice:** compare a direct model call, a fixed workflow, and the agent on the same task set and budget. Inject harmless tool failures; log recovery, loop counts, and stopping behavior. Treat external documents as data rather than tool instructions.
- **Self-check:** separate task failure, tool failure, unsupported answers, and budget exhaustion. Add multiple agents only as an optional ablation after establishing a single-agent baseline.

14 · Capstone: a GenAI system for a statistician

Choose one modest project: a citation-grounded assistant for public statistical notes, a teaching assistant that explains simulation results, or a tool-using simulation planner with a constrained action space.

Produce a **reproducible Python notebook and a short MyST chapter** containing:

1. A target user, task population, loss, and explicit scope.
2. A simple baseline and one justified extension, such as RAG or tool use.
3. Separate development and held-out evaluation sets, with provenance.
4. Repeated-run results, task-level uncertainty, cost, and failure examples.
5. An ablation that tests what the added component contributes.
6. A plain-language explanation of what the evidence supports and where the system fails.

Success means being able to **explain, implement, evaluate, and teach** the system. A small, well-understood experiment is a complete capstone.

Optional directions after the core

Multimodal models, advanced diffusion/SDE theory, scaling laws, interpretability, reasoning-model training, causal questions about evaluation, and multi-agent systems can each become a later topic. Select them to answer a concrete research question rather than trying to learn every framework at once.

Learn, teach, and build with Python

The whole website is built by a **Python toolchain: Sphinx + MyST-NB + sphinx-book-theme**. Python executes the lesson cells, makes the figures, and builds the static HTML pages. You can publish that HTML on GitHub Pages. The browser uses the theme's bundled CSS and JavaScript for navigation and search; authoring and building require neither Node.js nor Quarto.

1 · Choose how to study

- **Read first:** all examples and their outputs are visible on the website.
- **Experiment:** download the Chapter 1 notebook[9], open it in JupyterLab, and change the code.
- **Write lessons:** edit the Markdown files under `docs/` and rebuild the website.

A *terminal* accepts commands that manage files and run programs. A *notebook* contains text and Python code cells. Enter the commands below in a terminal; enter the Python examples from the lessons in notebook code cells. The shell prompt is not part of a command.

2 · Create a Python environment

Use **Python 3.12** for the tested course environment. A virtual environment keeps this course's packages separate from other projects.

Download or clone Teaching-GenAI on GitHub[10], then open a terminal in that folder. If Git is installed:

```
git clone https://github.com/chikuang/Teaching-GenAI.git
cd Teaching-GenAI
```

On macOS or Linux, with Python 3.12 installed as `python3.12`:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

On Windows, in PowerShell:

[9] https://chikuang.github.io/teaching/genai/notebooks/01-probability-to-genai.ipynb
[10] https://github.com/chikuang/Teaching-GenAI

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Here `-m` runs a Python module. `venv` creates the environment; activation makes `python` refer to that environment; `pip install` installs the packages listed in the file. Run `python --version` to check the interpreter. If PowerShell blocks activation, use `.venv\Scripts\python.exe` wherever the instructions say `python`; activation is a convenience, not a requirement.

`requirements.txt` gives compatible package ranges. `requirements-lock.txt` records the tested macOS/Python 3.12 environment; a platform-specific package in that snapshot may need adjustment on another operating system. The first chapter runs on a CPU with no API key or model download.

3 · Open a notebook

Save the downloaded notebook in the course folder, then run:

```
python -m jupyterlab
```

JupyterLab opens in your browser. Open the `.ipynb` file using its file browser. Select a code cell and press **Shift+Enter**. Run from the top: later cells refer to earlier objects. A number in brackets indicates a completed execution; an asterisk indicates a running cell.

To check reproducibility, choose **Kernel → Restart Kernel and Run All Cells**. The downloaded notebook already contains the website's executed outputs, but you can edit the cells and run them again. Saving a notebook preserves its code and outputs; it does not preserve every object in the running kernel's memory.

4 · Build the website and PDF

From the repository root, with the environment active:

```
python scripts/build.py
```

This command produces both the HTML website and the complete course PDF from the same lesson sources, like rendering a Quarto book to HTML and PDF. It exports the notebooks, executes new or changed notebook content in fresh kernels, and reuses those successful outputs in both formats. It stops on execution errors, Sphinx warnings, or a PDF compilation failure.

PDF generation requires **XeLaTeX and latexmk** in addition to the Python packages. Install MacTeX on macOS or TeX Live on Linux. On Ubuntu, the required packages are:

```
sudo apt-get install latexmk texlive-xetex texlive-latex-extra texlive-fonts-
↪recommended fonts-freefont-otf
```

Successful output is written to `_build/html/`, with executed notebooks under `notebooks/` and the complete book at `STAT-XXXX-Generate-AI.pdf`. The PDF contains a linked contents page, mathematical formulas, code, executed results, and figures. The **PDF** button at the top of every course page downloads the book.

To regenerate only the PDF:

```
python scripts/build.py --format pdf
```

This updates `_build/html/STAT-XXXX-Generate-AI.pdf` without rebuilding the HTML pages. The intermediate LaTeX source and compilation logs are in `_build/latex/`.

Download the published course PDF[11].

To preview the result:

```
python -m http.server 8767 --directory _build/html
```

Open `http://localhost:8767` in a browser. Keep the terminal running while you browse. Press **Control+C** to stop the server. After editing a lesson, run the build command again and refresh the browser. This is an explicit rebuild, not automatic live reload.

5 · Understand the source files

Path	Purpose
`docs/index.md`	Homepage and chapter navigation.
`docs/chapters/`	Canonical lesson text and Python code cells.
`docs/conf.py`	Python configuration for Sphinx and notebook execution.
`docs/_static/course.css`	White backgrounds, readable typography, and yellow/pink accents.
`scripts/build.py`	Build HTML/PDF, execute lessons, and export notebooks.
`requirements.txt`	Python package dependencies.
`_build/html/`	Generated website; rebuild it rather than editing it.

The lessons use **MyST Markdown**, which combines ordinary Markdown, mathematical notation, and executable notebook cells. A minimal executable lesson looks like this:

````
---
file_format: mystnb
kernelspec:
  name: python3
  display_name: Python 3
  language: python
---

# A small probability example

For counts $n_k$, estimate $p_k$ by $n_k / N$.

```{code-cell} python
import numpy as np
counts = np.array([6, 3, 1])
probabilities = counts / counts.sum()
print(probabilities)
```
````

[11] https://chikuang.github.io/teaching/genai/STAT-XXXX-Generate-AI.pdf

The first block identifies a notebook and its kernel. The Markdown becomes prose; the `code-cell` block is executed by Python during the build. Add a new chapter to the `toctree` in `docs/index.md` so that it appears in the navigation. The build script discovers executable MyST chapters automatically and exports their notebooks.

Use MyST-NB's authoring documentation[12] for notebook syntax and Sphinx's documentation[13] for site configuration. Chapter 1's notebook uses standard Markdown and LaTeX so its prose can also be read directly in JupyterLab.

6 · Publish a reviewed build

The public course lives at chikuang.github.io/teaching/genai/[14]. Its source is in `chikuang/Teaching-GenAI`; rendered pages live in the separate `chikuang/chikuang.github.io` repository under `teaching/genai/`.

After building and previewing, copy the **contents** of `_build/html/` into that course directory, then commit and push the reviewed source and rendered pages to their respective repositories. Keep the course directory separate from the rest of the personal website. GitHub Pages serves the published HTML.

The course's GitHub Actions workflow independently executes and builds the lessons and saves an artifact containing both HTML and the course PDF. It validates the source; publication to the personal website remains a separate step.

[12] https://myst-nb.readthedocs.io/en/latest/authoring/text-notebooks.html
[13] https://www.sphinx-doc.org/en/master/
[14] https://chikuang.github.io/teaching/genai/

The statistical lens

For each new idea, ask: **What are the observations? What is random? What is the target? What is being optimized? What assumptions justify the evaluation?** The connections below are learning aids; their limits matter as much as their similarities.

A translation guide

GenAI concept	Statistical starting point	What needs care
Language modeling	Conditional density estimation and the chain rule	Token likelihood measures text prediction, not the truth of a claim.
Cross-entropy training	Maximum likelihood and empirical risk minimization	Token observations within documents are dependent; split and evaluate at an appropriate unit.
Embeddings	Learned feature maps and low-dimensional representations	Coordinates are learned representations, not uniquely identified scientific factors.
Attention	Query-dependent weighted averages, reminiscent of kernel smoothing	Queries, keys, and values are learned; attention weights are not automatically causal importance or posterior probabilities.
Transformer	A composition of learned feature transformations	Attention is one component alongside positional information, nonlinear layers, and residual connections.
Temperature	A transformation of a categorical sampling distribution	More or less randomness does not by itself improve calibration or factual correctness.
Variational autoencoder	Latent-variable modeling and variational inference	The encoder approximates a posterior under the fitted model; approximation and model misspecification both matter.
Diffusion	Denoising, score estimation, and stochastic processes	A learned reverse process approximates a distribution; it does not recover a unique original observation from noise.
LoRA	A low-rank restriction on parameter updates	This is a constraint on an update matrix, not necessarily on the entire fitted model.
Preference optimization	Paired-comparison models and regularized decision making	Annotator preferences and reward proxies are not objective truth.
RAG	Retrieval followed by prediction conditional on selected evidence	Retrieved documents are selected, potentially incomplete or wrong, and usually not random samples from a posterior.
Agent	Sequential decisions with actions, observations, and a stopping rule	The language model is only one part of the system; tools, environment, and accumulated errors also matter.
LLM evaluation	Measurement, experimental design, and uncertainty quantification	Repeated answers to one prompt do not create independent test tasks.

Three equations to keep in view

A language model is a distribution over sequences

For tokens $x_1, \ldots, x_T$,

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Training commonly minimizes the sum of negative log conditional probabilities. Generation then samples from a conditional distribution, often modified by a decoding rule. Training and decoding answer different questions.

Attention forms a data-dependent weighted average

For one attention head,

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \qquad Z = AV,$$

where softmax is applied row by row and M can mask unavailable positions. With $Q \in \mathbb{R}^{n_q \times d_k}$, $K \in \mathbb{R}^{n_k \times d_k}$, and $V \in \mathbb{R}^{n_k \times d_v}$, the output is $Z \in \mathbb{R}^{n_q \times d_v}$. Each row of A sums to one over allowed positions. This invites a comparison to a normalized smoother, while the learned representations and end-to-end training make it different from a fixed kernel estimator. See Vaswani et al.[15].

Evaluate a system on a specified task distribution

$$R(\pi) = \mathbb{E}_{\tau \sim \mathcal{D}}[L(\pi, \tau)]\,.$$

Here π is the full system, τ a task, and L a loss that may include failure, cost, and latency. Specify the task population and sampling design before reporting a performance estimate. For stochastic systems, distinguish between-task variation from variation across repeated runs of the same task.

Questions for every paper or experiment

1. What is the estimand or target behavior, and what is actually measured?
2. What are the training, validation, and test units? Where could leakage enter?
3. Which quantities are learned, fixed, sampled, or selected adaptively?
4. What simple baseline would tell us whether the additional complexity helps?
5. How would distribution shift, dependence, or misspecification change the conclusion?
6. Which claims are empirically supported, and which remain useful analogies?

[15] https://arxiv.org/abs/1706.03762

Reading guide

Start with the introductory resources. Read foundational papers after working through the corresponding small example. This is a selected learning path, not an exhaustive survey or a claim about the newest model.

First-pass resources

Resource	When to use it	Reading goal
Dive into Deep Learning[16]	Modules 2–5	Work through arrays, autodiff, multilayer perceptrons, attention, and transformers; skip familiar mathematical preliminaries as needed.
PyTorch: Learn the Basics[17]	Module 2 onward	Learn tensors, datasets, model definitions, gradients, and the training loop.
Hugging Face LLM Course[18]	Modules 3, 5, 6, and 9	Connect tokenization and transformer concepts to practical model tooling.
MyST-NB[19] and Sphinx[20]	Throughout	Write and build executable teaching notes with Python.

Foundational papers by module

- **Attention and transformers — Module 4–5:** Vaswani et al. (2017), Attention Is All You Need[21]. First identify the inputs, outputs, attention equation, and block diagram; return to experiments later.
- **Latent-variable generation — Module 7:** Kingma and Welling (2013 preprint), Auto-Encoding Variational Bayes[22]. Connect the ELBO to variational inference and study the reparameterization step.
- **Diffusion — Module 8:** Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models[23]. Track the forward and reverse processes and the denoising objective.

[16] https://d2l.ai/
[17] https://docs.pytorch.org/tutorials/beginner/basics/intro.html
[18] https://huggingface.co/learn/llm-course/chapter1/1
[19] https://myst-nb.readthedocs.io/en/latest/
[20] https://www.sphinx-doc.org/
[21] https://arxiv.org/abs/1706.03762
[22] https://arxiv.org/abs/1312.6114
[23] https://arxiv.org/abs/2006.11239

- **Efficient adaptation — Module 9:** Hu et al. (2021 preprint), LoRA: Low-Rank Adaptation of Large Language Models[24]. Identify which parameters are frozen and which low-rank matrices are learned.
- **Preference learning — Module 9:** Rafailov et al. (2023), Direct Preference Optimization: Your Language Model is Secretly a Reward Model[25]. Study the preference likelihood and the role of the reference policy.
- **Retrieval — Module 10:** Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[26]. Identify the retriever, generator, and conditioning information; distinguish the original model from a simple modern retrieval pipeline.
- **Uncertainty — Module 11:** Angelopoulos and Bates (2021 preprint), A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification[27]. Establish the prediction target, calibration set, and coverage assumptions before applying the ideas to generated outputs.
- **Agents — Module 12:** Yao et al. (2022 preprint; ICLR 2023), ReAct: Synergizing Reasoning and Acting in Language Models[28]. Follow the interaction with tools and observations, and inspect how success is measured.

A statistician's reading note

For each paper, write a short note answering: What is the statistical object? What is optimized? What assumptions are made? What is the comparison baseline? What are the experimental units? Which claim would I want to reproduce?

Links and source identities were checked when this outline was created on October 20, 2026. Software documentation evolves; record the versions used for each future lab.

[24] https://arxiv.org/abs/2106.09685
[25] https://arxiv.org/abs/2305.18290
[26] https://arxiv.org/abs/2005.11401
[27] https://arxiv.org/abs/2107.07511
[28] https://arxiv.org/abs/2210.03629

1 · From probability models to generative AI

Complete lesson · Python from the beginning · A statistician's perspective

Download the executed notebook[29] · *Set up Python* (page 10)

You already know how to define a model, estimate its parameters, assess its fit, and simulate from it. These are useful starting points for generative AI. The unfamiliar part is often the data representation and software vocabulary, rather than the basic idea of learning a distribution.

By the end of this chapter you will fit a categorical distribution, generate observations from it, and build a small language model that generates short sequences. You will also understand why a model can assign high probability to a plausible sentence without establishing that the sentence is true.

The examples use a laptop CPU, NumPy, and Matplotlib. No GPU, pretrained model, API key, or prior Python knowledge is required. Run code cells **from top to bottom**: later cells use objects defined earlier. The website displays the outputs from a successful clean execution. To change a cell and run it yourself, use the downloadable notebook in JupyterLab or VS Code.

1.1 · What is a generative model?

Let X be an observation and P_0 its unknown data-generating distribution. A statistical model specifies a family $\{P_\theta : \theta \in \Theta\}$. After observing training data D, we obtain an estimate $\widehat{\theta}$ or a posterior distribution for θ.

There are several different things we might do next:

Operation	Statistical question	Example
Estimation	What parameter or function of the distribution is supported by the data?	Estimate a categorical probability vector.
Prediction	What distribution should we assign to an unobserved outcome?	Predict the next word given the preceding words.
Generation	How can we produce a new realization from the fitted distribution?	Draw a word from that categorical distribution.
Evaluation	How well does the fitted system perform on a specified population of tasks?	Measure held-out log loss on new sentences.

For example, fitting a normal distribution and calling a random-number generator already gives a simple gen-

[29] https://chikuang.github.io/teaching/genai/notebooks/01-probability-to-genai.ipynb

erative procedure. Modern generative AI uses richer representations and models to generate objects such as text, images, audio, or code. The distribution may be explicit, as in a next-token model, or defined through a sampling mechanism without an easily evaluated density.

The distinction between *generative* and *discriminative* needs context. In classical classification, a generative classifier models $p(x, y)$ while a discriminative classifier models $p(y \mid x)$. In GenAI, a system can generate an entire response y conditional on a prompt c using $p_\theta(y \mid c)$. Conditional modeling and generation are therefore compatible.

A useful separation: the model specifies probabilities; a decoding algorithm decides how to turn them into outputs. Sampling, choosing the most probable outcome, and modifying probabilities before sampling are different procedures even when they use the same fitted model.

A small vocabulary

Term	Meaning in this course
Token	A unit in a sequence: a character, word, word fragment, or special symbol. Our first language model uses words.
Vocabulary	The finite set of tokens the model can produce.
Context	Information available when predicting the next token.
Parameter	A fitted quantity, such as a categorical probability or a neural network weight.
Training	Fitting the parameters using data.
Inference in ML software	Often running a fitted model to obtain predictions. This use of "inference" is narrower than statistical inference about unknown quantities.
Decoding	Turning predicted token probabilities into an output sequence.
LLM	A large language model. The small count model in this chapter is a language model, but it is not an LLM.

1.2 · The statistical workflow we will preserve

We will use the sequence

$$\text{data} \longrightarrow \text{representation} \longrightarrow \text{fitted distribution} \longrightarrow \text{prediction or generation} \longrightarrow \text{evaluation}.$$

Different errors can occur at each step. A vocabulary can exclude important outcomes; an estimator can overfit; a sampler can produce an unlikely realization; and an evaluation set can fail to represent the intended population. Improving the neural architecture would not automatically fix all of these problems.

Three quantities must stay distinct:

1. **Population distribution P_0:** what produced the observations, usually unknown.
2. **Fitted distribution $P_{\widehat{\theta}}$:** our model after training.
3. **Empirical distribution of generated draws:** a Monte Carlo approximation to the fitted distribution, conditional on the fitted model.

If a million generated draws accurately reproduce $P_{\widehat{\theta}}$, we have checked a sampling computation. We have not established that $P_{\widehat{\theta}}$ is close to P_0.

1.3 · The Python you need to start

Values, names, and lists

In a notebook, enter a code cell and press **Shift+Enter** to execute it. A notebook keeps objects in memory, so executing cells out of order can produce confusing results. Use "Restart Kernel and Run All" when checking that a notebook is reproducible.

```
sample_size = 10
course_name = "STAT XXXX Generate AI"
words = ["data", "models", "learning"]

print(course_name)
print(sample_size)
print(words[0])
print(len(words))
```

```
STAT XXXX Generate AI
10
data
3
```

Read this one line at a time:

- `sample_size = 10` assigns the integer `10` to a name. `=` is assignment; `==` tests equality.
- Text inside quotation marks is a **string**. Either single or double quotes can delimit a string.
- Square brackets create a **list**, an ordered collection of values.
- `words[0]` selects the first element. Python counts positions from zero: the valid indices here are `0`, `1`, and `2`.
- `len(words)` calls the built-in function `len` and returns the number of items.
- `print(...)` displays a value. Parentheses contain the function's inputs.

A name does not declare a permanent type. Although Python permits reassigning `sample_size` to a string, doing so would make the code harder to follow. Use descriptive names and keep their meaning stable.

Numerical arrays with NumPy

A Python list is a general container. A NumPy array is designed for numerical operations on vectors and matrices.

```
import numpy as np

counts = np.array([6, 3, 1])
total = counts.sum()
p_hat = counts / total

print("Counts:", counts)
print("Total:", total)
```

(continues on next page)

(continued from previous page)

```
print("Estimated probabilities:", p_hat)
print("Array shape:", counts.shape)
```

```
Counts: [6 3 1]
Total: 10
Estimated probabilities: [0.6 0.3 0.1]
Array shape: (3,)
```

`import numpy as np` loads the NumPy library and gives it the short name `np`. The dot in `np.array` accesses a function supplied by that library. The dot in `counts.sum()` accesses an array's method. A **method** is a function attached to an object. `counts.shape` is an attribute, so it has no parentheses.

The expression `counts / total` divides every entry by the same scalar. This is **broadcasting**. The shape `(3,)` means a one-dimensional array of length three. It is not explicitly a 3×1 column matrix. We will inspect shapes carefully when we reach attention.

For a Python list, `[1, 2] * 2` repeats the list. For a NumPy array, multiplication by `2` multiplies each number. In NumPy, `*` is elementwise multiplication and `@` is matrix multiplication. Never infer the operation from the mathematical role you intended; inspect the object type and operator.

Indexing, loops, and numerical checks

```
print("First probability:", p_hat[0])
print("First two probabilities:", p_hat[:2])
print("Last probability:", p_hat[-1])

for word, probability in zip(words, p_hat):
    print(word, round(float(probability), 3))

assert np.all(p_hat >= 0)
assert np.isclose(p_hat.sum(), 1.0)
```

```
First probability: 0.6
First two probabilities: [0.6 0.3]
Last probability: 0.1
data 0.6
models 0.3
learning 0.1
```

`p_hat[:2]` selects entries starting at zero and stopping **before** index two. A negative index counts backward from the end. `zip(words, p_hat)` pairs the first word with the first probability, and so on. The `for` loop takes one pair at a time. Its indented lines form the loop body; indentation is part of Python syntax.

`float` converts a NumPy scalar to an ordinary Python floating-point number. `round(..., 3)` controls the displayed precision here; it does not change `p_hat`. `np.all` checks that every comparison is true. `np.isclose` allows a small floating-point tolerance instead of expecting exact arithmetic. An `assert` stops execution if its condition is false. These checks are part of understanding the calculation.

Checkpoint. Before moving on, predict what `counts[1]`, `counts[:1]`, and `counts.sum()` return. Answers: `3`, an array containing `6`, and `10`.

1.4 · Fit a categorical model by maximum likelihood

Suppose $Y_1, \ldots, Y_N$ take values in a vocabulary of K categories. Write $n_j = \sum_i \mathbf{1}\{Y_i = j\}$ and $p_j = P(Y = j)$. Under an independent categorical model,

$$L(p) = \prod_{i=1}^{N} p_{Y_i} = \prod_{j=1}^{K} p_j^{n_j}, \qquad \ell(p) = \sum_{j=1}^{K} n_j \log p_j.$$

Maximize this expression subject to $p_j \geq 0$ and $\sum_j p_j = 1$. For positive counts, a Lagrange multiplier gives

$$\frac{\partial}{\partial p_j}\left[\sum_k n_k \log p_k + \lambda\left(1 - \sum_k p_k\right)\right] = \frac{n_j}{p_j} - \lambda = 0.$$

Summing $p_j = n_j/\lambda$ yields $\lambda = N$, so $\widehat{p}_j = n_j/N$. Zero-count categories receive zero mass in the boundary solution. The omitted multinomial coefficient would not change the maximizer if we started from the likelihood of the count vector.

We have already implemented this estimator with `counts / counts.sum()`. No machine learning library is needed to see the fitting operation.

From observations to counts

So far, counts were given to us. Now we encode observations by category index and count them.

```
train_ids = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 2])
estimated_counts = np.bincount(train_ids, minlength=len(words))
p_mle = estimated_counts / estimated_counts.sum()

print("Counts recovered from observations:", estimated_counts)
print("MLE:", p_mle)
assert np.array_equal(estimated_counts, counts)
```

```
Counts recovered from observations: [6 3 1]
MLE: [0.6 0.3 0.1]
```

`np.bincount` counts occurrences of nonnegative integer IDs. `minlength=3` asks for at least three output positions, including a zero if a category is unobserved. This keyword argument makes the intended vocabulary size explicit. The probability vector and the vocabulary must use the same ordering; swapping labels while keeping probabilities fixed silently changes the model.

1.5 · Log loss, cross-entropy, and perplexity

To evaluate a probability assignment q, score each observed outcome by $-\log q_{Y_i}$. The average is

$$\widehat{R}(q) = -\frac{1}{n}\sum_{i=1}^{n} \log q_{Y_i}.$$

This is empirical cross-entropy, or average negative log likelihood. With natural logarithms, the unit is **nats per observation**. Maximizing training likelihood is equivalent to minimizing training log loss. The important change for evaluation is to use observations that did not determine the fitted model or its tuning choices.

Here is a reusable function:

```
def mean_log_loss(probabilities, observed_ids):
    assigned = probabilities[observed_ids]
    if np.any(assigned <= 0):
        return np.inf
    return -np.log(assigned).mean()

test_ids = np.array([0, 1, 2, 2, 1, 0])
uniform = np.full(len(words), 1 / len(words))

for label, candidate in [("Fitted", p_mle), ("Uniform", uniform)]:
    loss = mean_log_loss(candidate, test_ids)
    print(f"{label}: loss = {loss:.3f}; perplexity = {np.exp(loss):.3f}")
```

```
Fitted: loss = 1.339; perplexity = 3.816
Uniform: loss = 1.099; perplexity = 3.000
```

`def` defines a function; its arguments are names available inside its body. `probabilities[observed_ids]` performs multiple lookups at once, returning the probability assigned to each observed category. `np.any` checks whether at least one condition holds. `return` sends a result back to the caller and exits the function. `np.inf` represents positive infinity.

`np.log` applies the natural logarithm entry by entry. `.mean()` averages the results. The minus sign turns log probabilities into nonnegative losses. The final line uses an **f-string**: the expressions inside braces are evaluated and `:.3f` displays three digits after the decimal point. The pairs in the loop are two-element **tuples**.

On these intentionally tiny data, the uniform model has lower held-out loss than the fitted model. This is possible even though the MLE minimizes training loss. The training sample is more concentrated on category zero than the test sample. One small, manually constructed test set cannot establish a general ranking.

Why is log loss statistically meaningful?

For a true categorical distribution p_0 and a candidate q with positive mass wherever p_0 is positive,

$$\mathbb{E}_{p_0}[-\log q(Y)] = H(p_0) + D_{\mathrm{KL}}(p_0 \| q).$$

Thus the population expected loss is minimized at $q = p_0$. This statement concerns the probability target and the evaluation population. It does not imply that a low-loss language model answers factual questions accurately: the outcome being predicted is an observed token, not a truth label.

Perplexity is $\exp(\widehat{R})$ when natural logs are used. A uniform distribution on K outcomes has perplexity K. General perplexity is an effective uncertainty scale, not a literal count of equally likely words. Comparisons require compatible tokenization, vocabulary conventions, datasets, and averaging rules; a word-level score and a subword score are not directly interchangeable.

1.6 · Zero counts, smoothing, and a Bayesian connection

An unseen category gets zero probability under the categorical MLE. If it later occurs, its log loss is infinite. This exposes an assumption: zero observed count has been treated as evidence of impossibility.

A simple alternative is

$$\widetilde{p}_j = \frac{n_j + \alpha}{N + K\alpha}, \qquad \alpha > 0.$$

Under a symmetric Dirichlet$(\alpha, \ldots, \alpha)$ prior and a multinomial model, this is both the posterior mean of p_j and the one-step posterior predictive probability. It is generally **not the posterior mode**. It is also a shrinkage estimate:

$$\widetilde{p}_j = \frac{N}{N + K\alpha}\widehat{p}_j + \frac{K\alpha}{N + K\alpha}\frac{1}{K}.$$

```
sparse_counts = np.array([4, 1, 0])
alpha = 0.5
p_unsmoothed = sparse_counts / sparse_counts.sum()
p_smoothed = (sparse_counts + alpha) / (sparse_counts.sum() + alpha * 3)

print("Unsmoothed:", p_unsmoothed)
print("Smoothed:", np.round(p_smoothed, 3))
print("Loss of unseen category, unsmoothed:", mean_log_loss(p_unsmoothed, np.
↪array([2])))
print("Loss of unseen category, smoothed:", mean_log_loss(p_smoothed, np.array([2])))
```

```
Unsmoothed: [0.8 0.2 0. ]
Smoothed: [0.692 0.231 0.077]
Loss of unseen category, unsmoothed: inf
Loss of unseen category, smoothed: 2.5649493574615367
```

Adding a scalar to an array adds it to every count. The denominator adds the same total pseudo-count mass, preserving a sum of one. The value $\alpha = 0.5$ is fixed in advance for this lesson. In an empirical study, select it using training/validation data or a justified prior; do not choose it after examining final test performance.

Smoothing prevents a particular failure but does not solve model misspecification. If the scientific vocabulary omitted an entire class of outcomes, adding pseudo-counts to the remaining categories cannot recover it.

1.7 · Generation is a sampling operation

Once probabilities are fitted, we can generate observations. This is conceptually separate from fitting them.

```
rng = np.random.default_rng(2026)
generated_ids = rng.choice(len(words), size=12, p=p_mle)
generated_words = [words[i] for i in generated_ids]

print("Generated IDs:", generated_ids)
print("Generated words:", " ".join(generated_words))
```

```
Generated IDs: [0 1 0 0 0 1 2 0 1 0 2 2]
Generated words: data models data data data models learning data models data learning␣
↪learning
```

`default_rng(2026)` creates a pseudorandom generator initialized by a seed. Its `.choice` method draws from the integer IDs `0`, `1`, and `2`, using `p=p_mle`. The keyword `size=12` requests twelve draws. Sampling is with replacement by default.

The expression `[words[i] for i in generated_ids]` is a **list comprehension**: take each generated ID, look up its word, and collect the results. `" ".join(...)` joins strings with spaces. We have generated a bag-like stream of independent words. There is no grammar or memory in this model.

Check the sampler with Monte Carlo

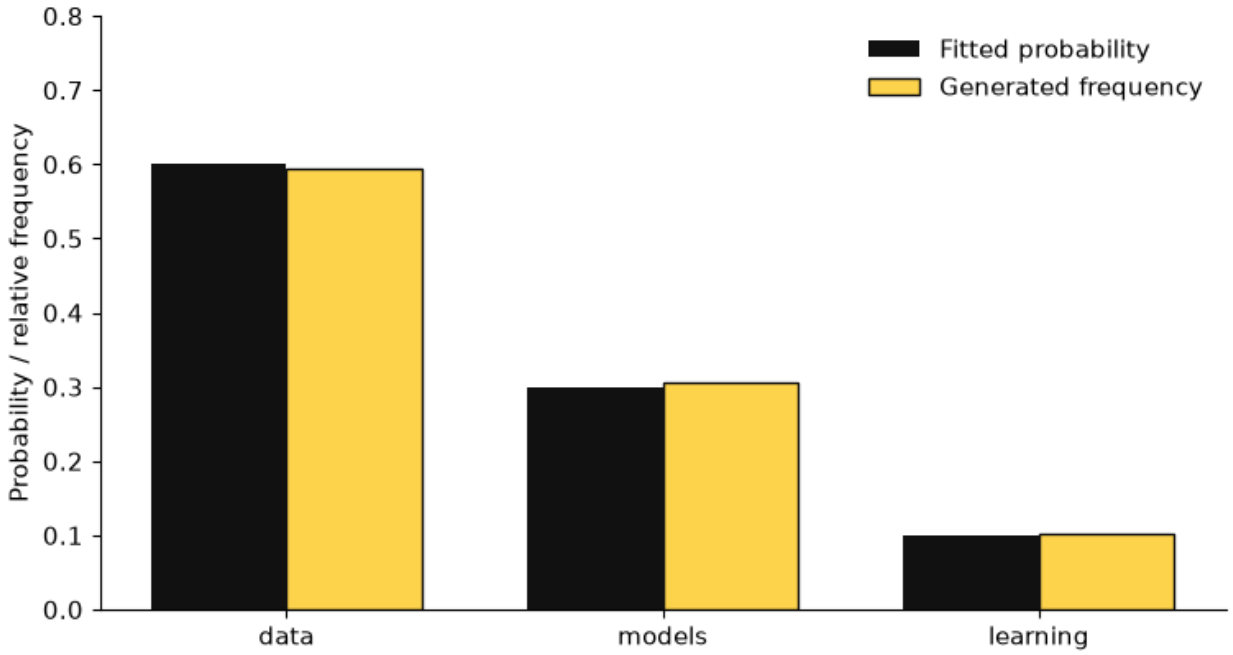
```
import matplotlib.pyplot as plt

plt.rcParams.update({"figure.figsize": (7.5, 4), "font.size": 11})
rng_check = np.random.default_rng(21)
large_sample = rng_check.choice(len(words), size=10_000, p=p_mle)
frequencies = np.bincount(large_sample, minlength=len(words)) / len(large_sample)
mc_se = np.sqrt(p_mle * (1 - p_mle) / len(large_sample))

print("Target:", p_mle)
print("Simulated frequencies:", np.round(frequencies, 4))
print("Marginal Monte Carlo SEs:", np.round(mc_se, 4))

x = np.arange(len(words))
fig, ax = plt.subplots(layout="constrained")
ax.bar(x - 0.18, p_mle, width=0.36, color="#111111", label="Fitted probability")
ax.bar(x + 0.18, frequencies, width=0.36, color="#FED34C", edgecolor="#111111", label=
↪"Generated frequency")
ax.set_xticks(x, words)
ax.set_ylabel("Probability / relative frequency")
ax.set_ylim(0, 0.8)
ax.legend(frameon=False)
ax.spines[["top", "right"]].set_visible(False)
plt.show()
```

```
Target: [0.6 0.3 0.1]
Simulated frequencies: [0.5935 0.3052 0.1013]
Marginal Monte Carlo SEs: [0.0049 0.0046 0.003 ]
```



`plt.subplots` creates a figure and an axes object. `ax.bar` adds bars; the shifts `-0.18` and `+0.18` place each pair side by side. `np.arange(3)` creates positions `0, 1, 2`. `set_xticks` labels those positions, and `plt.show()` displays the figure.

The Monte Carlo standard errors are conditional on the fitted probability vector and assume independent draws. They do not include uncertainty from estimating that vector. Multinomial category frequencies are also correlated, so these marginal standard errors are not simultaneous uncertainty statements. A seed supports reproducibility; it does not eliminate either kind of uncertainty.

1.8 · Temperature changes the distribution used for sampling

Many generation interfaces offer a **temperature** parameter. For finite logits z_j and $T > 0$,

$$q_T(j) = \frac{\exp(z_j/T)}{\sum_k \exp(z_k/T)}.$$

If $z_j = \log p_j$ with all $p_j > 0$, then $q_T(j) \propto p_j^{1/T}$. At $T = 1$ we recover p. Smaller T concentrates mass on the largest probabilities. As $T \to \infty$, the distribution approaches uniformity over this finite support. As $T \downarrow 0$, mass concentrates on maximizers; ties require care. Greedy decoding chooses a maximizer directly and is a distinct algorithm.

```python
def temperature_distribution(probabilities, temperature=1.0):
    if temperature <= 0:
        raise ValueError("Temperature must be positive.")
    if np.any(probabilities <= 0):
        raise ValueError("This example requires positive probabilities.")
    scaled_logits = np.log(probabilities) / temperature
    weights = np.exp(scaled_logits - scaled_logits.max())
    return weights / weights.sum()
```

(continues on next page)

(continued from previous page)

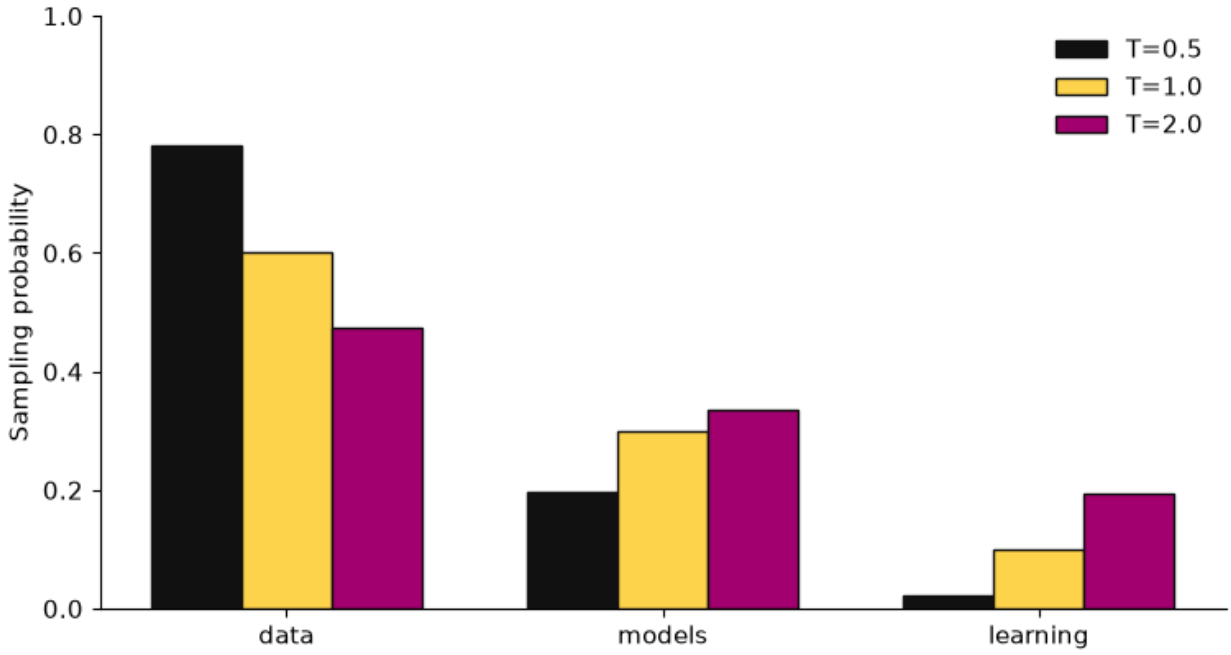
```
temperatures = [0.5, 1.0, 2.0]
for temperature in temperatures:
    q = temperature_distribution(p_mle, temperature)
    entropy = -np.sum(q * np.log(q))
    print(f"T={temperature:.1f}: probabilities={np.round(q, 3)}, entropy={entropy:.3f}
↪")

assert np.allclose(temperature_distribution(p_mle, 1.0), p_mle)
```

```
T=0.5: probabilities=[0.783 0.196 0.022], entropy=0.594
T=1.0: probabilities=[0.6 0.3 0.1], entropy=0.898
T=2.0: probabilities=[0.473 0.334 0.193], entropy=1.038
```

`raise ValueError(...)` reports an invalid input. The default argument `temperature=1.0` means the caller may omit it. Subtracting the largest scaled logit before exponentiating leaves the normalized probabilities unchanged and helps prevent overflow. `np.allclose` checks every element with a tolerance.

```
fig, ax = plt.subplots(layout="constrained")
colors = ["#111111", "#FED34C", "#A2006E"]
for index, temperature in enumerate(temperatures):
    q = temperature_distribution(p_mle, temperature)
    ax.bar(x + (index - 1) * 0.24, q, width=0.24,
           color=colors[index], edgecolor="#111111", label=f"T={temperature}")
ax.set_xticks(x, words)
ax.set_ylabel("Sampling probability")
ax.set_ylim(0, 1)
ax.legend(frameon=False)
ax.spines[["top", "right"]].set_visible(False)
plt.show()
```



`enumerate` supplies both a loop counter and the current value. The counter chooses a color and a horizontal offset. The pink bars highlight the flatter distribution at $T = 2$.

Temperature changes the generated distribution without changing the trained parameters. Increased diversity is not the same as improved predictive accuracy, uncertainty calibration, or factuality. If a model is confidently wrong, making its output less random can make it consistently wrong.

1.9 · From independent tokens to a language model

Independent token sampling discards order. For a sequence $x_1, \ldots, x_T$, the probability chain rule gives

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

The chain rule is an identity; it does not assume a Markov process. A **bigram model** adds the approximation

$$p_\theta(x_t \mid x_{<t}) = p_\theta(x_t \mid x_{t-1}).$$

The previous token is the entire context. This is an explicit first-order Markov assumption. A transformer later replaces this small conditioning rule with a learned function of a longer context. The basic next-token probability and sequential sampling interpretation remain useful.

We add special symbols: `<bos>` marks the beginning, and `<eos>` marks the end. For "data support models," our scored prediction events are

$$\texttt{<bos>} \to \texttt{data}, \quad \texttt{data} \to \texttt{support}, \quad \texttt{support} \to \texttt{models}, \quad \texttt{models} \to \texttt{<eos>}.$$

The start symbol is a context, not an output token. Including the end symbol makes stopping part of the probability model. In our smoothed finite-state example every context gives positive probability to `<eos>`, so the process eventually stops with probability one. We will still impose a length cap in software.

Make a small, transparent corpus

```
train_sentences = [
    "data support models",
    "models explain data",
    "data guide learning",
    "learning improves models",
    "models generate data",
    "data support learning",
]
test_sentences = [
    "models support learning",
    "data explain models",
    "learning generate data",
]

def tokenize(sentence):
    return sentence.lower().split()

print(tokenize(train_sentences[0]))
```

```
['data', 'support', 'models']
```

This corpus was written for the lesson, not sampled from a population of natural language. Awkward test sentences are intentional combinations for inspecting the model, not a benchmark of linguistic quality. We split by complete sentences before counting transitions so adjacent tokens from the same sentence do not leak across partitions.

`sentence.lower()` lowercases a string, and `.split()` separates it at whitespace. Real tokenizers handle punctuation, subwords, and many other cases; our authored sentences avoid punctuation. Changing a tokenizer changes the modeled outcome space.

Build a vocabulary and lookup tables

```
training_words = []
for sentence in train_sentences:
    training_words.extend(tokenize(sentence))

lexical_vocabulary = sorted(set(training_words))
output_vocabulary = lexical_vocabulary + ["<eos>"]
contexts = ["<bos>"] + lexical_vocabulary

word_to_id = {word: index for index, word in enumerate(output_vocabulary)}
context_to_id = {word: index for index, word in enumerate(contexts)}

print("Output vocabulary:", output_vocabulary)
print("Index of 'data':", word_to_id["data"])
print("Number of output categories:", len(output_vocabulary))
```

```
Output vocabulary: ['data', 'explain', 'generate', 'guide', 'improves', 'learning',
↪'models', 'support', '<eos>']
Index of 'data': 0
Number of output categories: 9
```

`extend` adds each item from a list to another list. `set` removes duplicates; `sorted` places them in a deterministic order. `+` concatenates Python lists. A **dictionary** maps keys to values: `word_to_id["data"]` retrieves the integer ID associated with a string. The expression `{word: index for ...}` is a dictionary comprehension, analogous to the list comprehension used earlier.

Only training data determine the vocabulary. The hand-written test set uses words already present in training. A real application needs a specified strategy for unseen words, such as a training-defined `<unk>` symbol or a fixed subword tokenizer. Do not expand the vocabulary using the final test data merely to make evaluation convenient.

Count transitions and estimate conditional distributions

```
transition_counts = np.zeros((len(contexts), len(output_vocabulary)), dtype=int)

for sentence in train_sentences:
    sequence = ["<bos>"] + tokenize(sentence) + ["<eos>"]
    for previous, following in zip(sequence[:-1], sequence[1:]):
```

(continues on next page)

(continued from previous page)

```
        row = context_to_id[previous]
        column = word_to_id[following]
        transition_counts[row, column] += 1

alpha = 0.5
row_totals = transition_counts.sum(axis=1, keepdims=True)
transition_probabilities = (transition_counts + alpha) / (
    row_totals + alpha * len(output_vocabulary)
)

assert np.allclose(transition_probabilities.sum(axis=1), 1.0)
print("Transition count matrix shape:", transition_counts.shape)
print("Number of training prediction events:", transition_counts.sum())
```

```
Transition count matrix shape: (9, 9)
Number of training prediction events: 24
```

`np.zeros((r, c), dtype=int)` creates an integer matrix with r rows and c columns. `sequence[:-1]` omits the last item; `sequence[1:]` omits the first. Zipping the two produces consecutive pairs. `transition_counts[row, column] += 1` increments the selected entry.

`axis=1` sums across columns, producing one total per context. `keepdims=True` retains a column-shaped result, so each matrix row is divided by its own normalizing constant through broadcasting. Without tracking these shapes, a calculation can fail or normalize along the wrong dimension.

Statistically, the model is a separate categorical distribution for each context. Its likelihood groups transition counts by row. Adding α yields row-wise posterior predictive probabilities under independent symmetric Dirichlet priors. This is a small conditional density estimator, not a neural network.

```
data_row = context_to_id["data"]
for token, probability in zip(output_vocabulary, transition_probabilities[data_row]):
    print(f"P({token:>8} | data) = {probability:.3f}")

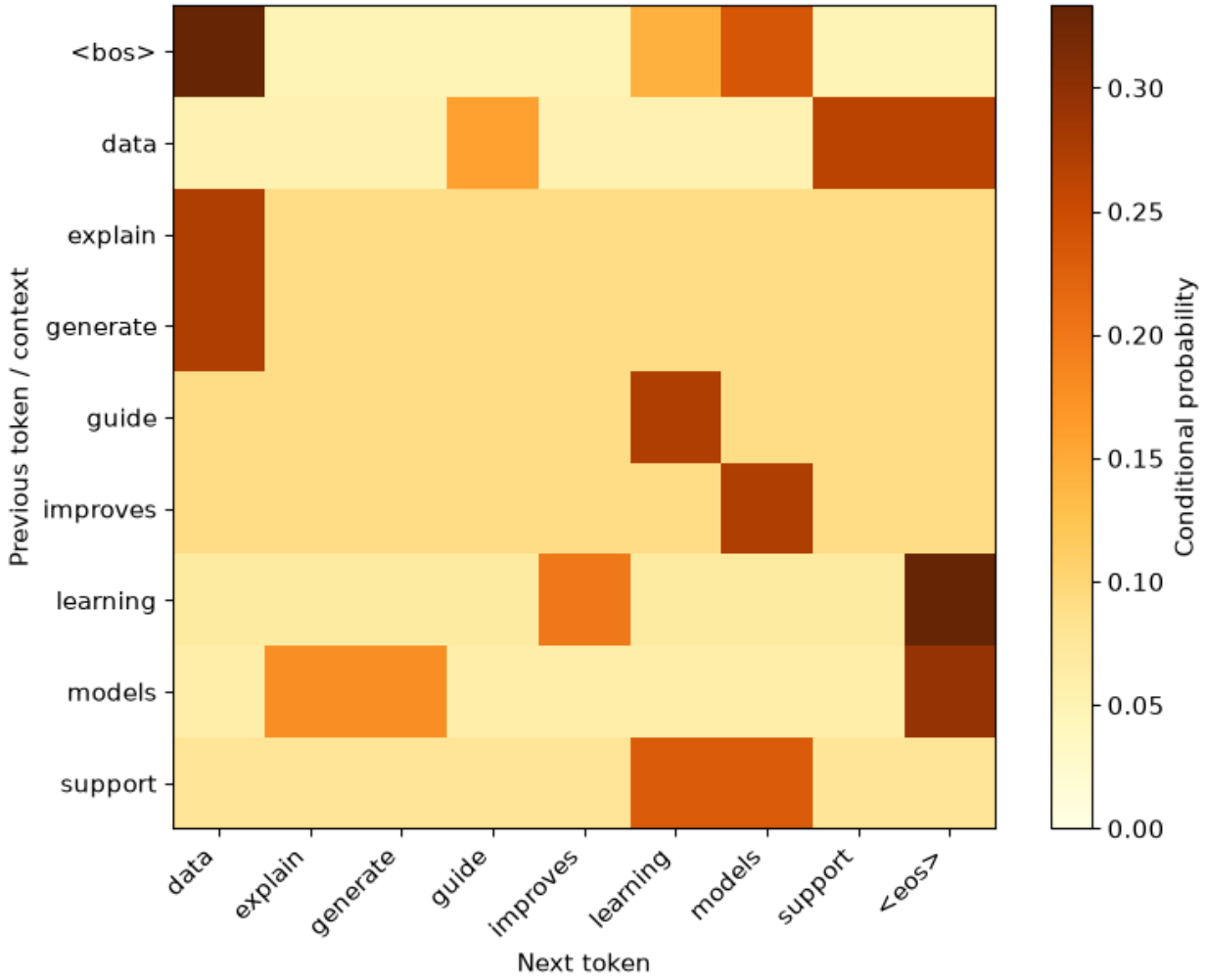
fig, ax = plt.subplots(figsize=(8, 6), layout="constrained")
image = ax.imshow(transition_probabilities, cmap="YlOrBr", vmin=0,
                  vmax=transition_probabilities.max())
ax.set_xticks(np.arange(len(output_vocabulary)), output_vocabulary, rotation=45, ha=
↪"right")
ax.set_yticks(np.arange(len(contexts)), contexts)
ax.set_xlabel("Next token")
ax.set_ylabel("Previous token / context")
fig.colorbar(image, ax=ax, label="Conditional probability")
plt.show()
```

```
P(    data | data) = 0.053
P( explain | data) = 0.053
P(generate | data) = 0.053
P(   guide | data) = 0.158
P(improves | data) = 0.053
P(learning | data) = 0.053
P(  models | data) = 0.053
```

(continues on next page)

(continued from previous page)

```
P( support | data) = 0.263
P(   <eos> | data) = 0.263
```



The format `:>8` right-aligns a token in a field of width eight. `imshow` displays a matrix as an image; the colorbar translates shading back to probabilities. The rows sum to one; columns generally do not. A heatmap is useful only if you know which axis represents the conditioning event.

1.10 · Generate a sequence one decision at a time

```python
def generate_sentence(probability_table, rng, max_tokens=15):
    previous = "<bos>"
    result = []
    for step in range(max_tokens):
        row = context_to_id[previous]
        next_id = rng.choice(len(output_vocabulary), p=probability_table[row])
        next_word = output_vocabulary[next_id]
        if next_word == "<eos>":
```

(continues on next page)

(continued from previous page)

```
            return " ".join(result), "eos"
        result.append(next_word)
        previous = next_word
    return " ".join(result), "length cap"

rng_text = np.random.default_rng(8670)
for repetition in range(5):
    sentence, stop_reason = generate_sentence(transition_probabilities, rng_text)
    print(f"{repetition + 1}. {sentence!r} [{stop_reason}]")
```

```
1. 'data' [eos]
2. '' [eos]
3. 'data guide learning support improves generate support support' [eos]
4. 'learning data generate generate' [eos]
5. 'learning' [eos]
```

`range(15)` produces integers zero through fourteen. The loop samples one next token per iteration, appends it to a list, and updates the context. `append` adds a single item. `return` can return a tuple; the caller unpacks it into `sentence` and `stop_reason`. `!r` displays a string with quotes, making an empty generated sentence visible.

The length cap is an operational stopping rule, not an additional learned token. A sequence stopped by the cap has been truncated; it is not the same event as a model-generated `<eos>`. The printed stop reason prevents that distinction from being hidden.

These outputs may repeat, stop immediately, or be ungrammatical. Additive smoothing allows every next token from every context, and a one-token context cannot represent long-range syntax. That is an observation about our fitted model. The random generator is doing the sampling job we asked it to do.

This code repeatedly samples from a **fixed table** of smoothed probabilities. It does not integrate a shared uncertain transition matrix over the entire generated path. A fully Bayesian sequence posterior predictive would account for dependence induced by that shared parameter uncertainty. The plug-in sampler is sufficient for understanding the first algorithm, provided we name what it computes.

1.11 · Evaluate prediction separately from generation

We evaluate probabilities assigned to observed test tokens, using their actual preceding tokens as contexts. We do not condition on the model's previously sampled tokens during this calculation. In neural language modeling, training on the observed prefix is commonly called **teacher forcing**.

```
def bigram_log_loss(sentences, probability_table):
    losses = []
    for sentence in sentences:
        sequence = ["<bos>"] + tokenize(sentence) + ["<eos>"]
        for previous, following in zip(sequence[:-1], sequence[1:]):
            row = context_to_id[previous]
            column = word_to_id[following]
            losses.append(-np.log(probability_table[row, column]))
    return float(np.mean(losses))
```

(continues on next page)

(continued from previous page)

```
unigram_counts = transition_counts.sum(axis=0)
unigram_probabilities = (unigram_counts + alpha) / (
    unigram_counts.sum() + alpha * len(output_vocabulary)
)
unigram_table = np.tile(unigram_probabilities, (len(contexts), 1))

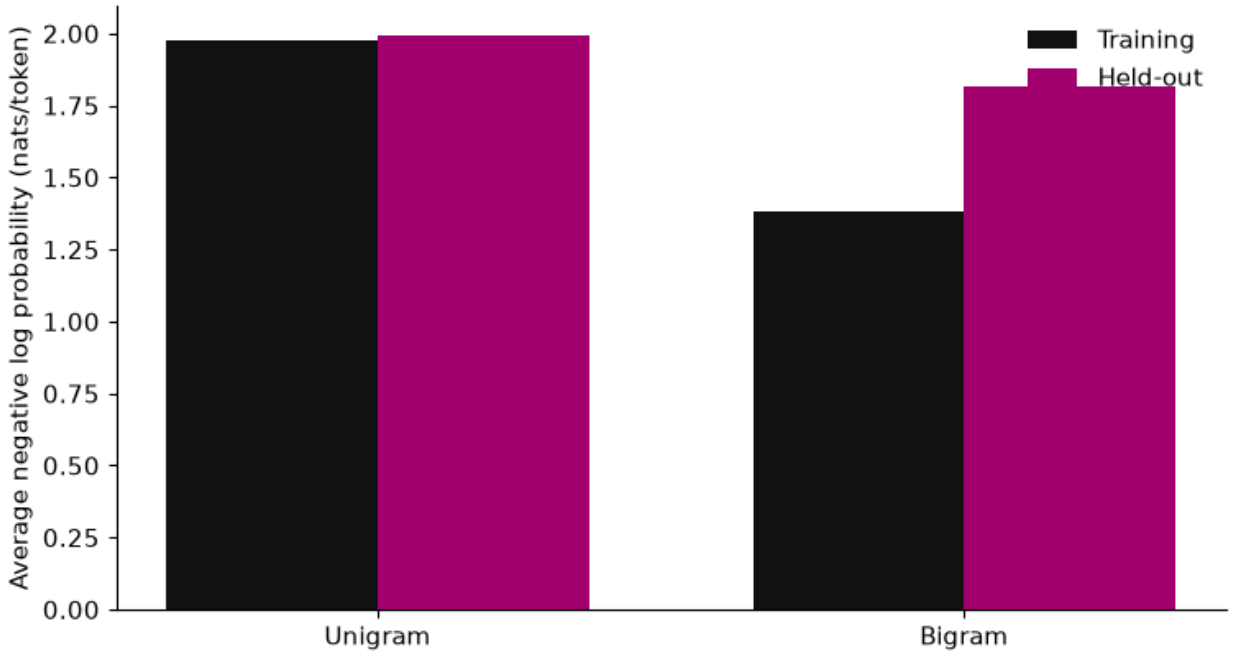
evaluation = {}
for label, table in [("Unigram", unigram_table), ("Bigram", transition_
↪probabilities)]:
    train_loss = bigram_log_loss(train_sentences, table)
    test_loss = bigram_log_loss(test_sentences, table)
    evaluation[label] = (train_loss, test_loss)
    print(f"{label}: train={train_loss:.3f}, test={test_loss:.3f}, test perplexity=
↪{np.exp(test_loss):.3f}")
```

```
Unigram: train=1.976, test=1.994, test perplexity=7.348
Bigram: train=1.381, test=1.816, test perplexity=6.144
```

`axis=0` sums down the context rows, leaving a count for each output token. `np.tile` repeats the same unigram vector in every row; this constructs a baseline that ignores the context while using the same scoring function and vocabulary. Both models include `<eos>` in their prediction events and use the same fixed smoothing parameter.

The dictionary `evaluation` stores a pair of results for each model. The function pools all prediction events before averaging: longer sentences contribute more terms. An equal-weight average of sentence-level average losses would be a different estimand. Decide which target you want before comparing models.

```
fig, ax = plt.subplots(layout="constrained")
positions = np.arange(2)
train_losses = [evaluation[name][0] for name in ["Unigram", "Bigram"]]
test_losses = [evaluation[name][1] for name in ["Unigram", "Bigram"]]
ax.bar(positions - 0.18, train_losses, width=0.36, color="#111111", label="Training")
ax.bar(positions + 0.18, test_losses, width=0.36, color="#A2006E", label="Held-out")
ax.set_xticks(positions, ["Unigram", "Bigram"])
ax.set_ylabel("Average negative log probability (nats/token)")
ax.legend(frameon=False)
ax.spines[["top", "right"]].set_visible(False)
plt.show()
```



Compare the outputs before drawing a conclusion. A conditional model can improve training fit while generalizing poorly to different transition patterns. The toy corpus is much too small and deliberately constructed to support a scientific claim of superiority. The purpose is to make the distinction between training fit and held-out prediction observable.

For a real study, define the target population, reserve a test set, choose smoothing on a separate validation set, and compare systems on the same test documents. Tokens within a document are dependent. Treating thousands of token losses as independent observations can give misleadingly narrow intervals. Resampling documents or another justified independent cluster is often more defensible; the number of independent clusters still matters.

Predictive probability is not factual confidence

Our model learns which tokens follow others in the corpus. It has no variable representing whether "models explain data" is scientifically correct in a particular context. A larger model's next-token objective also does not itself supply a calibrated probability that an entire answer is true. Evaluating factuality requires a defined factual target, evidence, and an appropriate measurement procedure. Later chapters develop retrieval and evaluation in response to this gap.

1.12 · Common Python problems and how to diagnose them

Symptom	Likely cause	First action
`NameError: name 'np' is not defined`	The import cell has not run in this kernel.	Restart and run all cells in order.
`ModuleNotFoundError`	The selected kernel does not have the package.	Confirm the notebook uses the course environment.
`IndexError`	A position is outside the array/list.	Print its length or shape and inspect the index.
`KeyError` for a token	The token is absent from the dictionary.	Inspect the vocabulary and tokenization; specify unknown-token handling.
`IndentationError`	A loop/function body is not consistently indented.	Use four spaces and check the enclosing block.
Probabilities do not sum to one	The wrong axis or denominator was used.	Print row totals and check `axis` and `keepdims`.
Different output after rerunning only one cell	A generator's internal state has advanced.	Recreate the seeded generator or restart and run all.

Do not fix `log(0)` by silently clipping probabilities and then reporting the score as if it came from the original model. If you change the probabilities, document the changed model and normalization. Likewise, rounding is for display; premature rounding can destroy a probability sum or a numerical comparison.

1.13 · Exercises

Attempt the questions before reading the worked solutions. You can use the same notebook; keep experimental changes in separate cells so the main example remains reproducible.

Exercise 1 · Read a small Python calculation

For `a = np.array([2, 5, 3])`, predict the values and shapes of `a[1]`, `a[:2]`, `a / a.sum()`, and `a * 2`. Explain how `[2, 5, 3] * 2` differs.

Exercise 2 · Estimate, smooth, and interpret

For counts $(8, 2, 0)$, derive the categorical MLE and the smoothed estimator with $\alpha = 1$. Compute the log loss if the next observation is category three. Is the smoothed estimate a posterior mean, posterior mode, or both under a Dirichlet$(1, 1, 1)$ prior?

Exercise 3 · Separate sampling error from estimation error

Generate 100, 1,000, and 10,000 observations from the fixed vector $(0.6, 0.3, 0.1)$. Compare the first-category frequency with its target. Explain why a single run need not become closer at every sample size. What experiment would be required to include uncertainty in the original fitted vector?

Exercise 4 · Understand temperature mathematically

Show that for two positive base probabilities,

$$\frac{q_T(i)}{q_T(j)} = \left(\frac{p_i}{p_j}\right)^{1/T}.$$

Compute the ratio when $p_i = 0.6$, $p_j = 0.3$, and $T \in \{0.5, 1, 2\}$. What happens if the base probabilities are tied? Does the ratio tell you whether either token is true?

Exercise 5 · Audit one transition row

Using the training corpus, count the transitions following `data` by hand, including end-of-sentence transitions. With $\alpha = 0.5$, compute the probabilities of `support`, `<eos>`, and `generate` given `data`. Check them against the matrix.

Exercise 6 · Score a whole sentence

For "data support models," write its probability as a product of four terms including `<eos>`. Implement the log probability as a sum. Explain why averaging over four events differs from averaging only over the three lexical words.

Exercise 7 · Design an honest comparison

You try twenty smoothing parameters and report the one with the lowest score on `test_sentences`. Identify the problem, propose a training/validation/test procedure, and state an appropriate resampling unit if sentences are grouped within documents.

Exercise 8 · A small extension with a baseline

Create twelve new, original sentences using a small vocabulary. Reserve complete sentences for validation and testing before fitting. Compare unigram and bigram models with the same tokenizer, support, and scoring convention. Report a failure example and one limitation. Do not interpret the small exercise as a reliable estimate of general language performance.

1.14 · Worked solutions

Solution 1

The scalar `a[1]` is `5`; `a[:2]` is an array `[2, 5]` with shape `(2,)`; normalization gives `[0.2, 0.5, 0.3]` with shape `(3,)`; and array multiplication gives `[4, 10, 6]`. Python list multiplication repeats items, producing `[2, 5, 3, 2, 5, 3]`.

Solution 2

The MLE is $(0.8, 0.2, 0)$; smoothing gives $(9/13, 3/13, 1/13)$. The loss on category three changes from infinity to $\log 13 \approx 2.565$ nats. The smoothed estimate is the posterior mean and one-step predictive vector. The posterior is Dirichlet$(9, 3, 1)$; its mode lies on the boundary, so it does not equal that mean.

```
exercise_counts = np.array([8, 2, 0])
exercise_prediction = (exercise_counts + 1) / (exercise_counts.sum() + 3)
print("Smoothed estimate:", exercise_prediction)
print("Loss on third category:", -np.log(exercise_prediction[2]))
```

```
Smoothed estimate: [0.69230769 0.23076923 0.07692308]
Loss on third category: 2.5649493574615367
```

Solution 3

```
rng_exercise = np.random.default_rng(7)
for n in [100, 1_000, 10_000]:
    draw = rng_exercise.choice(3, size=n, p=np.array([0.6, 0.3, 0.1]))
    first_frequency = np.mean(draw == 0)
    expected_se = np.sqrt(0.6 * 0.4 / n)
    print(f"n={n:5d}, frequency={first_frequency:.4f}, MC SE={expected_se:.4f}")
```

```
n=  100, frequency=0.6300, MC SE=0.0490
n= 1000, frequency=0.6060, MC SE=0.0155
n=10000, frequency=0.5921, MC SE=0.0049
```

`draw == 0` is a Boolean array; its mean is the proportion of `True` entries. The standard error decreases at rate $n^{-1/2}$, but realized deviations need not decrease monotonically. To include fitting uncertainty, repeatedly generate or resample training datasets under justified assumptions, refit the model, and then study predictions or generated summaries. A nested simulation can separate variability from fitting and from generation.

Solution 4

The normalization cancels in the ratio. The three ratios are 4, 2, and $\sqrt{2}$. Tied positive base probabilities remain tied at every positive temperature. These are probability ratios for modeled outcomes, with no independent statement about truth.

Solution 5

`data` has five outgoing events: `support` twice, `guide` once, and `<eos>` twice. There are nine output categories. The normalizer is $5 + 9(0.5) = 9.5$. Therefore the requested probabilities are $2.5/9.5$, $2.5/9.5$, and $0.5/9.5$.

```
row = context_to_id["data"]
for token, expected in [("support", 2.5 / 9.5), ("<eos>", 2.5 / 9.5), ("generate", 0.
↪5 / 9.5)]:
    actual = transition_probabilities[row, word_to_id[token]]
    assert np.isclose(actual, expected)
    print(token, round(float(actual), 4))
```

```
support 0.2632
<eos> 0.2632
generate 0.0526
```

Solution 6

The product is $p(\texttt{data} \mid \texttt{<bos>})p(\texttt{support} \mid \texttt{data})p(\texttt{models} \mid \texttt{support})p(\texttt{<eos>} \mid \texttt{models})$. Its logarithm is the sum of the corresponding four log probabilities.

```
sentence_events = [("<bos>", "data"), ("data", "support"),
                   ("support", "models"), ("models", "<eos>")]
log_probability = 0.0
for previous, following in sentence_events:
    log_probability += np.log(transition_probabilities[context_to_id[previous], word_
↪to_id[following]])
print("Sentence log probability:", round(float(log_probability), 4))
print("Sentence probability:", round(float(np.exp(log_probability)), 6))
assert np.isclose(-log_probability / 4, bigram_log_loss(["data support models"],␣
↪transition_probabilities))
```

```
Sentence log probability: -5.1237
Sentence probability: 0.005954
```

Leaving out <eos> scores the lexical prefix without scoring the decision to stop. The set of prediction events and denominator both change. Neither convention should be switched between competing systems without explanation.

Solution 7

The test set has become a tuning set. Split at the independent unit first; fit transition counts and vocabulary on training data; choose smoothing on validation data; then report the selected procedure's performance once on an untouched test set. If refitting on combined training and validation data, specify that step in advance. Resample independent documents rather than individual token losses when document-level dependence is the relevant structure. With few documents, uncertainty estimates remain unstable.

Solution 8 · Reporting checklist

Record the corpus, tokenizer, split, vocabulary/unknown-token rule, smoothing choices, and score definition. Show both models' losses on the same test events and several generated outputs, including failures and stop reasons. State whether the apparent gain could be due to chance, repeated tuning, or an unusually favorable split. A negative or inconclusive result is a valid result.

1.15 · What this chapter gives us

We have built a complete small modeling workflow: encode outcomes, fit probabilities, score held-out observations, sample new outcomes, and inspect the assumptions. The bigram table makes context-dependent prediction explicit. It also exposes the cost of a restricted context and sparse counts.

The next chapter replaces hand-estimated probability tables with differentiable functions. You will learn tensors and automatic differentiation through regression and a small neural predictor before using those tools for attention. Keep the same questions: what is the modeled distribution, what is optimized, and what evidence supports the result?

Further reading

- Python Tutorial[30]: basic values, lists, control flow, and functions.
- NumPy: the absolute basics for beginners[31]: arrays, indexing, shapes, and numerical operations.
- Speech and Language Processing, author-hosted draft[32]: the chapter on n-gram language models develops count-based sequence modeling further.

All data in this chapter are small, original teaching examples. Their role is to make a calculation inspectable, not to establish empirical claims about modern LLMs.

[30] https://docs.python.org/3/tutorial/
[31] https://numpy.org/doc/stable/user/absolute_beginners.html
[32] https://web.stanford.edu/~jurafsky/slp3/

2 · Python and neural network foundations

Chapter status: outline

This is the planned scope of Chapter 2. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. What changes when a regression model learns its own nonlinear features?

- **Learn:** arrays, shapes, indexing, broadcasting, functions, and reproducible random generators; then tensors, automatic differentiation, minibatches, and a PyTorch training loop.
- **Statistical lens:** linear/logistic regression, basis expansions, regularization, optimization, overfitting, and validation.
- **Python practice:** fit logistic regression and a small multilayer perceptron to the same simulated problem; record train/validation curves and compare against a simple baseline.
- **Self-check:** distinguish the model, loss, optimizer, and data split; check a derivative numerically before trusting autodiff.

3 · Tokens, embeddings, and representations

Chapter status: outline

This is the planned scope of Chapter 3. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How does text become something a numerical model can use?

- **Learn:** tokenization, vocabulary, one-hot representations, embedding lookup, positional information, cosine similarity, and contextual representations.
- **Statistical lens:** design matrices, learned feature maps, matrix factorization, and identifiability.
- **Python practice:** tokenize a small public text collection and compare bag-of-words with embedding-based similarity. Inspect examples where similarity is misleading.
- **Self-check:** explain why a token is not necessarily a word and why a two-dimensional embedding plot is not evidence of causal structure.

4 · Attention from first principles

Chapter status: outline

This is the planned scope of Chapter 4. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How can a representation select and combine information relevant to a query?

- **Learn:** queries, keys, values, scaled dot products, row-wise softmax, self-attention, cross-attention, causal masks, and multiple heads.
- **Statistical lens:** a query-dependent normalized weighted average; compare with kernel smoothing without claiming exact equivalence.
- **Python practice:** calculate a three-token attention example by hand, implement it in NumPy, verify dimensions and row sums, and plot how weights change when keys change.
- **Self-check:** explain the role of the mask and the $\sqrt{d_k}$ scaling; describe why attention weights alone are not a faithful explanation of a prediction.
- **Reading:** Attention Is All You Need[33], especially the architecture and attention equations.

[33] https://arxiv.org/abs/1706.03762

5 · Assembling the transformer

Chapter status: outline

This is the planned scope of Chapter 5. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How does one attention calculation become a sequence model?

- **Learn:** residual connections, normalization, feed-forward layers, positional encodings, encoder/decoder distinctions, and a decoder-only forward pass.
- **Statistical lens:** a composition of fitted feature transformations with shared parameters and a conditional prediction head.
- **Python practice:** implement a tiny decoder block in PyTorch; track every tensor's dimensions and use a causal-mask check to show future tokens cannot influence earlier outputs.
- **Self-check:** draw the route from token IDs to next-token probabilities, and distinguish the original encoder-decoder transformer from a decoder-only language model.

6 · Language model training, prompting, and decoding

Chapter status: outline

This is the planned scope of Chapter 6. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. What is learned in the weights, and what changes when we provide a prompt?

- **Learn:** next-token prediction, teacher forcing, context windows, pretraining versus inference, in-context examples, greedy decoding, temperature, top-k, and top-p.
- **Statistical lens:** conditional likelihood, distribution shift, and transformations of a sampling distribution. In-context behavior can be studied statistically without assuming the model performs exact Bayesian inference.
- **Python practice:** train a very small character model or use a small pretrained model; vary one decoding choice at a time on fixed prompts and repeat runs.
- **Self-check:** explain why perplexity is not a universal measure of usefulness and why prompting is not parameter estimation by gradient descent at inference time.

7 · Latent variables, VAEs, GANs, and flows

Chapter status: outline

This is the planned scope of Chapter 7. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. What alternatives to sequential factorization can represent a complex distribution?

- **Learn:** the encoder/decoder view of a VAE, the evidence lower bound (ELBO), reparameterization, and a conceptual comparison with adversarial learning and invertible flows.
- **Statistical lens:** latent-variable models, variational inference, density-ratio ideas, and change of variables. Compare objectives, sampling, and access to likelihoods.
- **Python practice:** derive the ELBO and implement a small VAE on synthetic two-dimensional data. Treat GANs and flows as comparison topics on the first pass.
- **Self-check:** separate the latent prior, approximate posterior, and observation model; explain why matching a few summary statistics cannot establish distributional equivalence.
- **Reading:** Auto-Encoding Variational Bayes[34].

[34] https://arxiv.org/abs/1312.6114

8 · Diffusion and score-based generation

Chapter status: outline

This is the planned scope of Chapter 8. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How can learning to remove noise support generation from noise?

- **Learn:** a forward noising process, learned reverse transitions, denoising objectives, score functions, and conditional generation; preview image and multimodal applications.
- **Statistical lens:** stochastic processes, denoising estimation, and the score $\nabla_x \log p(x)$, which differs from a likelihood score with respect to model parameters.
- **Python practice:** simulate progressive Gaussian noising on a mixture distribution, then study a small learned denoiser and the effect of the noise schedule.
- **Self-check:** distinguish the known forward process from the fitted reverse process and separate training from sampling.
- **Reading:** Denoising Diffusion Probabilistic Models[35]. Continuous-time formulations are an optional extension.

[35] https://arxiv.org/abs/2006.11239

9 · Fine-tuning, LoRA, and preference learning

Chapter status: outline

This is the planned scope of Chapter 9. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How do we adapt an existing model, and whose preferences are represented?

- **Learn:** supervised fine-tuning, parameter-efficient updates, LoRA, instruction data, preference data, the RLHF pipeline, and the basic DPO idea.
- **Statistical lens:** transfer learning, regularization, low-rank parameter updates, and Bradley–Terry-style paired comparisons. Keep model quality separate from preference-label quality.
- **Python practice:** fit a toy paired-comparison model and compare a full matrix update with a low-rank update. Small-model fine-tuning is optional after the conceptual exercise.
- **Self-check:** explain what changes under prompting, RAG, and fine-tuning; identify label noise and selection bias in preference data.
- **Readings:** LoRA[36] and DPO[37].

[36] https://arxiv.org/abs/2106.09685
[37] https://arxiv.org/abs/2305.18290

10 · Retrieval-augmented generation (RAG)

Chapter status: outline

This is the planned scope of Chapter 10. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. How can a model answer using a specified collection of documents?

- **Learn:** document chunking, indexing, retrieval, reranking, context assembly, citation, and abstention when evidence is insufficient.
- **Statistical lens:** prediction conditional on selected evidence, retrieval error, measurement error, and the interaction between pipeline components.
- **Python practice:** build a small pipeline over public statistical notes. Inspect retrieved passages, compare with a keyword baseline, and test answerable and unanswerable questions separately.
- **Self-check:** identify whether an error arose in retrieval, evidence selection, or generation. A citation does not by itself show that its source supports the claim.
- **Reading:** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[38].

[38] https://arxiv.org/abs/2005.11401

11 · Evaluation, calibration, and uncertainty

Chapter status: outline

This is the planned scope of Chapter 11. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. What evidence would justify saying one system is better than another?

- **Learn:** task-specific metrics, held-out evaluation, data contamination, human annotation, LLM-as-a-judge limitations, calibration, abstention, and an introduction to conformal methods.
- **Statistical lens:** estimands, sampling units, paired comparisons, bootstrap uncertainty, clustered dependence, and distribution shift. Conformal coverage needs an explicit target and assumptions such as exchangeability; it is not an automatic guarantee of free-text truthfulness.
- **Python practice:** compare two fixed systems on the same held-out questions; bootstrap at the question level, inspect judge disagreement, and draw a risk-versus-coverage curve for abstention.
- **Self-check:** distinguish token probabilities, answer confidence, and factual correctness; state the population to which a reported interval applies.
- **Reading:** A Gentle Introduction to Conformal Prediction[39].

[39] https://arxiv.org/abs/2107.07511

12 · Agents: tools, state, and decisions

Chapter status: outline

This is the planned scope of Chapter 12. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. What makes a system an agent, beyond producing a text response?

- **Learn:** a model–action–observation loop, tool schemas, state, memory, structured outputs, retries, and stopping rules; distinguish a fixed workflow from adaptive action selection.
- **Statistical lens:** sequential decision making, partial observation, expected utility, and the value of information. Learn the basic state/action/reward vocabulary without requiring a reinforcement learning course first.
- **Python practice:** build a single agent with two narrow tools, such as a calculator and document lookup. Validate arguments, log actions and observations, and enforce a small step budget.
- **Self-check:** explain when a fixed Python function or workflow is sufficient and why tool access does not imply reliable tool use.
- **Reading:** ReAct[40]; study observable actions and results without treating generated reasoning text as a faithful record of internal computation.

[40] https://arxiv.org/abs/2210.03629

13 · Agent reliability and experimental design

Chapter status: outline

This is the planned scope of Chapter 13. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Question. Does an agent improve outcomes enough to justify its additional complexity?

- **Learn:** trajectory-level evaluation, failure taxonomies, repeated trials, cost/latency tradeoffs, memory ablations, prompt injection through retrieved text, and human approval for consequential actions.
- **Statistical lens:** dependence within a task trajectory, factorial comparisons, uncertainty in success rates, and adaptive selection bias.
- **Python practice:** compare a direct model call, a fixed workflow, and the agent on the same task set and budget. Inject harmless tool failures; log recovery, loop counts, and stopping behavior. Treat external documents as data rather than tool instructions.
- **Self-check:** separate task failure, tool failure, unsupported answers, and budget exhaustion. Add multiple agents only as an optional ablation after establishing a single-agent baseline.

14 · Capstone: a GenAI system for a statistician

Chapter status: outline

This is the planned scope of Chapter 14. The complete lesson, executable examples, and solutions will be developed later. *Chapter 1* (page 19) is available now.

Choose one modest project: a citation-grounded assistant for public statistical notes, a teaching assistant that explains simulation results, or a tool-using simulation planner with a constrained action space.

Produce a **reproducible Python notebook and a short MyST chapter** containing:

1. A target user, task population, loss, and explicit scope.
2. A simple baseline and one justified extension, such as RAG or tool use.
3. Separate development and held-out evaluation sets, with provenance.
4. Repeated-run results, task-level uncertainty, cost, and failure examples.
5. An ablation that tests what the added component contributes.
6. A plain-language explanation of what the evidence supports and where the system fails.

Success means being able to **explain, implement, evaluate, and teach** the system. A small, well-understood experiment is a complete capstone.

First Python lab: probabilities to generation

This small experiment connects a familiar categorical model to the sampling step used in language generation. It runs on a CPU with NumPy and Matplotlib, with no model download or API account.

Download the Jupyter notebook

The web page displays results computed by Python when the course is built. To edit and run the cells yourself, open the downloaded notebook in JupyterLab or VS Code; the static web page does not run a Python kernel.

1 · Represent a probability distribution

Suppose a toy model predicts the next token from a vocabulary of three words. These probabilities are supplied for illustration; we have not trained a language model.

```
import numpy as np
import matplotlib.pyplot as plt

tokens = np.array(["statistics", "learning", "agents"])
p = np.array([0.60, 0.30, 0.10])
assert np.isclose(p.sum(), 1.0)
print(dict(zip(tokens.tolist(), p.tolist())))
```

```
{'statistics': 0.6, 'learning': 0.3, 'agents': 0.1}
```

`np.array` creates a numerical array. Python indexing starts at zero, so `p[0]` is the first probability. `np.isclose` compares floating-point values with a tolerance. `assert` stops execution if a check fails.

2 · Evaluate log loss

For observations $y_1, \ldots, y_n$ drawn from a categorical distribution, the average negative log likelihood is

$$\widehat{L}(p) = -\frac{1}{n}\sum_{i=1}^{n} \log p_{y_i}.$$

Here the observations are token indices. The array `p[observed]` looks up the predicted probability of each observed token.

```
observed = np.array([0, 0, 1, 0, 2, 1])
model_loss = -np.log(p[observed]).mean()
uniform = np.full(3, 1 / 3)
uniform_loss = -np.log(uniform[observed]).mean()
print(f"Toy model log loss: {model_loss:.3f} nats per token")
print(f"Uniform log loss:   {uniform_loss:.3f} nats per token")
```

```
Toy model log loss: 1.041 nats per token
Uniform log loss:   1.099 nats per token
```

This tiny sample is an illustration of the calculation, not evidence that one model will outperform another on a target population. In a real comparison, choose the model on development data and evaluate on separate held-out observations. Natural logarithms give losses in **nats**.

3 · Change the sampling temperature

Given logits z_j and temperature $T > 0$, the sampling probabilities are

$$p_T(j) = \frac{\exp(z_j/T)}{\sum_k \exp(z_k/T)}.$$

Taking $z_j = \log p_j$ recovers the original distribution at $T = 1$. Lower temperatures concentrate probability on higher-scoring tokens. Higher temperatures move this finite-vocabulary distribution toward uniformity. Neither operation establishes that a generated statement is true.

```
def softmax(logits, temperature=1.0):
    if temperature <= 0:
        raise ValueError("Temperature must be positive.")
    scaled = np.asarray(logits, dtype=float) / temperature
    weights = np.exp(scaled - scaled.max())
    return weights / weights.sum()

logits = np.log(p)
temperatures = [0.5, 1.0, 2.0]
probabilities = np.vstack([softmax(logits, t) for t in temperatures])
assert np.allclose(probabilities.sum(axis=1), 1.0)
assert np.allclose(softmax(logits, 1.0), p)
for t, row in zip(temperatures, probabilities):
    print(f"T={t:.1f}: {np.round(row, 3)}")
```

```
T=0.5: [0.783 0.196 0.022]
T=1.0: [0.6 0.3 0.1]
T=2.0: [0.473 0.334 0.193]
```

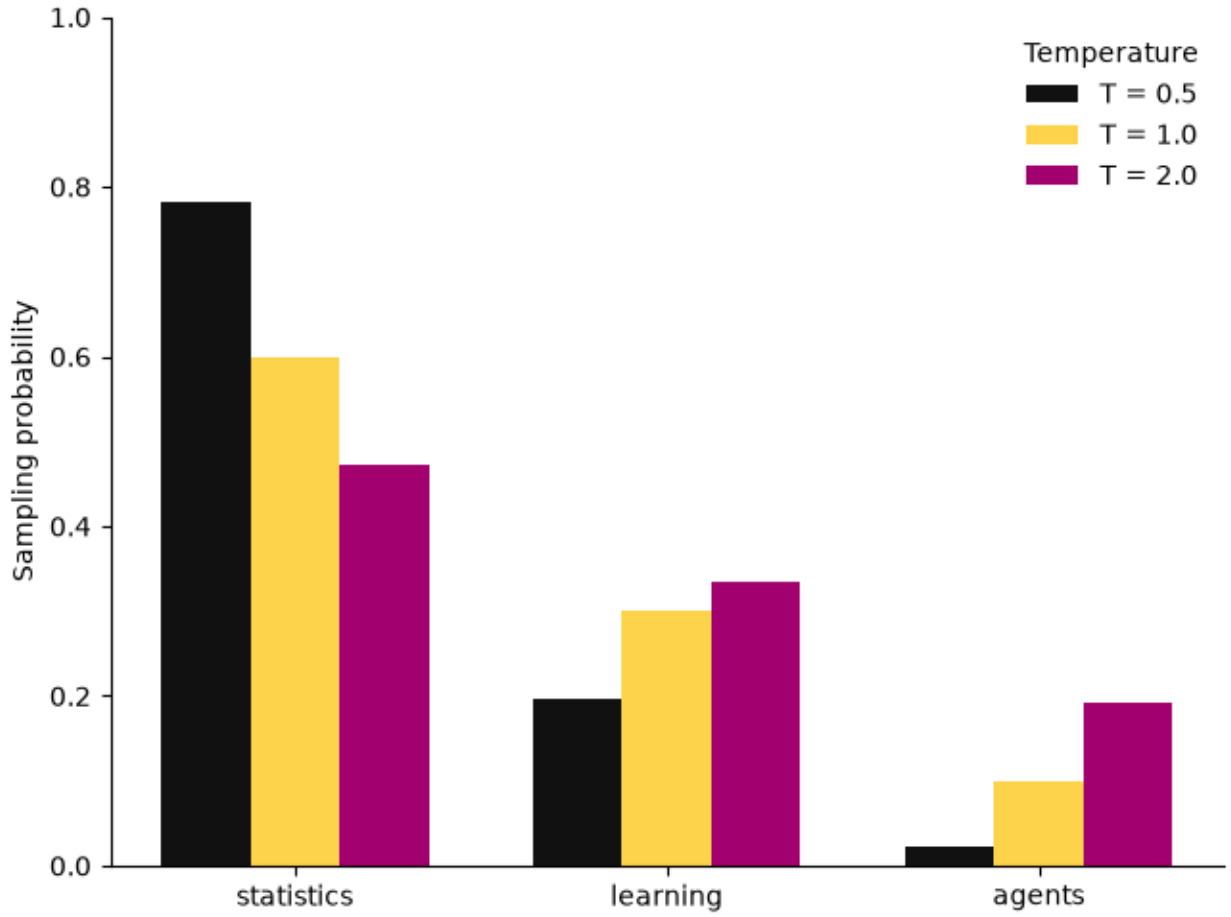
Subtracting the maximum before exponentiation leaves softmax unchanged and reduces overflow risk. A function is defined with `def`; indentation defines its body. The list comprehension computes one row for each temperature, and `np.vstack` combines the rows into a matrix.

```
fig, ax = plt.subplots(layout="constrained")
x = np.arange(len(tokens))
```

(continues on next page)

(continued from previous page)

```
width = 0.24
colors = ["#111111", "#FED34C", "#A2006E"]
for i, (t, row, color) in enumerate(zip(temperatures, probabilities, colors)):
    ax.bar(x + (i - 1) * width, row, width, label=f"T = {t}", color=color)
ax.set_xticks(x, tokens)
ax.set_ylabel("Sampling probability")
ax.set_ylim(0, 1)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(title="Temperature", frameon=False)
plt.show()
```



4 · Generate and inspect a Monte Carlo sample

```
rng = np.random.default_rng(8670)
draws = rng.choice(len(tokens), size=10_000, p=softmax(logits, 1.0))
frequencies = np.bincount(draws, minlength=len(tokens)) / len(draws)
print("Target:   ", p)
print("Observed: ", np.round(frequencies, 3))
print("One draw: ", tokens[draws[0]])
```

```
Target:    [0.6 0.3 0.1]
Observed:  [0.592 0.303 0.105]
One draw:  statistics
```

A fixed seed makes this example reproducible within the recorded environment. Sampling variation remains part of the statistical experiment. These independent draws are a toy example; an autoregressive language model instead updates its conditional distribution after each generated token.

Try it yourself

1. Change `p` while preserving positivity and a sum of one. Predict the temperature effect before running the code.
2. Repeat the Monte Carlo experiment with smaller sample sizes. Compare observed variability with the multinomial variance formula.
3. Try two different candidate distributions on the same observations. Explain why the lower observed loss alone is not a generalization guarantee.
4. Replace sampling by choosing the highest-probability token. Which aspect of the distribution disappears?

Bridge to attention

Softmax will return in attention, where it normalizes similarity scores into weights over available positions. The normalization is the same mathematical operation, but its role differs: attention weights combine value vectors, whereas the probabilities above sample an output token. The *statistical lens* (page 14) introduces the attention equation.